UNIVERSITY OF THE PEOPLE

CS 2203-01: Database Management Systems

AY2026-T4 Unit 2 Programming Assignment

Submitted by: Mubashir Ali

Instructor: John Onyianta

Date: April 18, 2026

Part (a): Entity Relationships and Key Attributes

This section defines the relationships between the four core entities in the e-commerce database Product, Customer, Order, and Inventory and identifies the key attributes for each.

Entity Attributes

1. Customer: Represents individuals who register and make purchases on the platform.

- CustomerID (Primary Key)
- FirstName, LastName
- Email
- Phone
- Address

2. Product: Represents items listed for sale on the e-commerce platform.

- ProductID (Primary Key)

- ProductName
- Category
- UnitPrice
- Description
- InventoryID (Foreign Key — references Inventory)

3. Order: Represents a purchase transaction made by a customer.

- OrderID (Primary Key)
- OrderDate
- TotalAmount
- Status
- CustomerID (Foreign Key - references Customer)
- ProductID (Foreign Key - references Product)

4. Inventory: Tracks stock levels for each product in the warehouse.

- InventoryID (Primary Key)
- ProductID (Foreign Key - references Product)
- StockQuantity
- ReorderLevel
- LastUpdated

Relationships Between Entities

Customer Order (PLACES): One Customer can place many Orders, but each Order belongs to exactly one Customer. This is a one-to-many (1:M) relationship. Customer participation is partial (not every customer has placed an order), while Order participation is total (every order must have a customer).

Order Product (CONTAINS): One Order can contain many Products, and one Product can appear in many Orders. This is a many-to-many (M:N) relationship. Both entities have partial participation since not every product appears in every order.

Product Inventory (TRACKED IN): Each Product has exactly one Inventory record, and each Inventory record corresponds to exactly one Product. This is a one-to-one (1:1) relationship with total participation on both sides.

Order Inventory (UPDATES): Each Order updates one or more Inventory records, while each Inventory record may be updated by many Orders. This is a one-to-many (1:M) relationship, ensuring stock levels are adjusted when orders are placed.

Part (b): Translation into Tables with Primary and Foreign Keys

The entities and attributes identified in Part (a) are now translated into relational database tables with appropriate primary and foreign keys, following the principles of the relational model.

Table 1: CUSTOMER

Column Name	Data Type	Constraint	Description
Customer ID	INT	PRIMARY KEY	Unique customer identifier
FirstName	VARCHAR (50)	NOT NULL	Customer first name
LastName	VARCHAR (50)	NOT NULL	Customer last name
Email	VARCHAR (100)	UNIQUE, NOT NULL	Customer email address
Phone	VARCHAR (20)	NULL	Contact phone number
Address	TEXT	NULL	Delivery address

Table 2: PRODUCT

Column Name	Data Type	Constraint	Description
ProductID	INT	PRIMARY KEY	Unique product identifier
ProductName	VARCHAR (100)	NOT NULL	Name of the product
Category	VARCHAR (50)	NOT NULL	Product category
UnitPrice	DECIMAL (10,2)	NOT NULL	Price per unit
Description	TEXT	NULL	Product description

InventoryID	INT	FOREIGN KEY	References Inventory (InventoryID)

Table 3: ORDER

Column Name	Data Type	Constraint	Description
OrderID	INT	PRIMARY KEY	Unique order identifier
OrderDate	DATE	NOT NULL	Date order was placed
TotalAmount	DECIMAL(12,2)	NOT NULL	Total cost of the order
Status	VARCHAR(30)	NOT NULL	e.g., Pending, Shipped
CustomerID	INT	FOREIGN KEY	References Customer(CustomerID)
ProductID	INT	FOREIGN KEY	References Product(ProductID)

Table 4: INVENTORY

Column Name	Data Type	Constraint	Description
InventoryID	INT	PRIMARY KEY	Unique inventory record ID
ProductID	INT	FOREIGN KEY	References Product (ProductID)
StockQuantity	INT	NOT NULL	Current units in stock
ReorderLevel	INT	NOT NULL	Minimum stock before reorder
LastUpdated	DATETIME	NOT NULL	Timestamp of last update

Part (c): Entity-Relationship (ER) Diagram

The ER diagram below visually represents the complete structure of the e-commerce database, incorporating all four entities, their attributes, and the relationships defined in Parts (a) and (b). Cardinality and participation constraints are clearly indicated using standard notation.

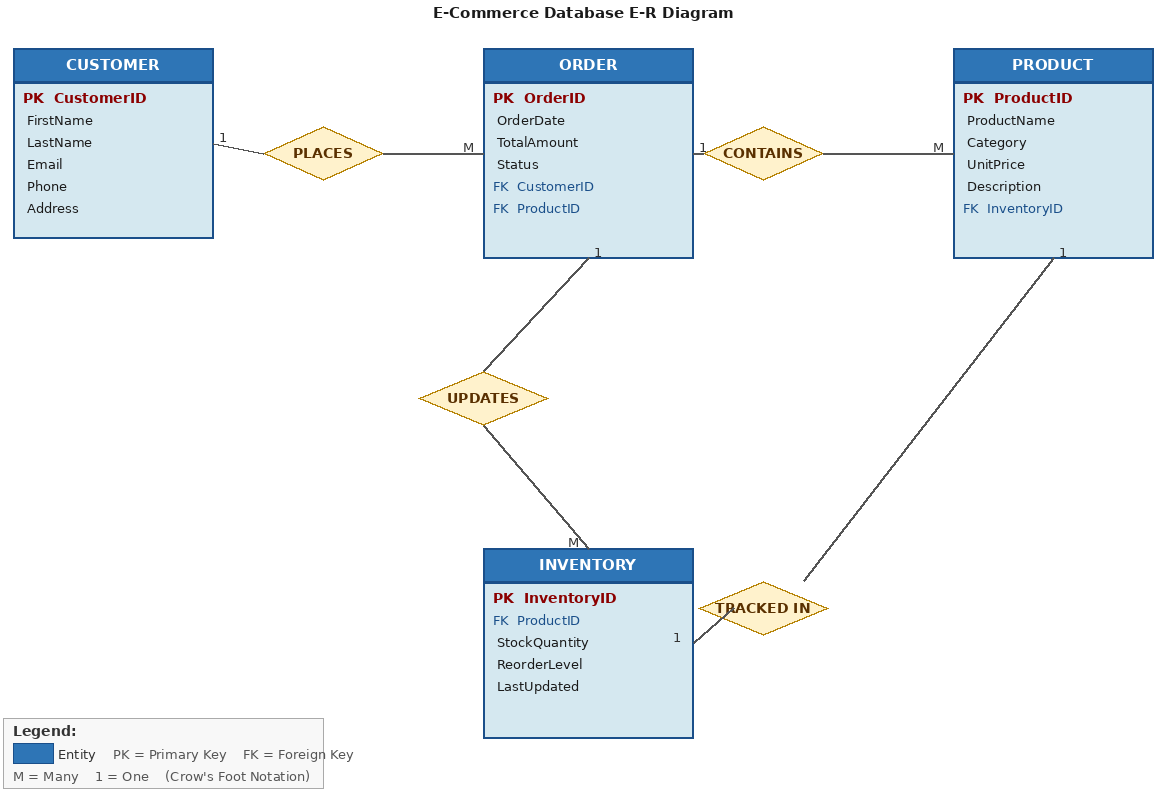


Figure 1: E-R Diagram for E-Commerce Database System

The diagram illustrates the following key relationships: Customer PLACES Order (1:M), Order CONTAINS Product (M:N), Product TRACKED IN Inventory (1:1), and Order UPDATES Inventory (1:M). Each entity box displays its primary key (PK) and foreign keys (FK) with all relevant attributes. The double-line connections indicate total participation, and single-lines indicate partial participation.

Part (d): Conceptual vs. Physical Design Two Key Differences

In the context of this e-commerce database, the conceptual and physical design models serve very different roles. The following two differences highlight their distinct contributions to database development.

Difference 1: Level of Abstraction vs. Technical Specificity

The conceptual design model operates at a high level of abstraction. In this e-commerce scenario, the conceptual model simply captures the existence of entities such as Customer, Order, Product, and Inventory, along with the nature of their relationships for example, that a customer places an

Order or that a Product is tracked in Inventory. There is no mention of data types, storage engines, or specific database software at this stage. The primary purpose is to communicate the business requirements clearly to both technical and non-technical stakeholders. In contrast, the physical design model is highly specific and technical. It defines exactly how each entity is stored in the database: for instance, specifying that CustomerID is an INT with AUTO_INCREMENT, that Email is VARCHAR(100) with a UNIQUE constraint, and that the relationship between Order and Customer is enforced using a FOREIGN KEY constraint referencing Customer(CustomerID). The physical design is tied to a specific DBMS such as MySQL or PostgreSQL, and it guides the actual construction of the database system.

Difference 2: Focus on Business Rules vs. Implementation Constraints

The conceptual design focuses on capturing business rules without concern for implementation limitations. For example, at the conceptual level, we express that a customer may place zero or many Orders, reflecting the real-world business rule of the e-commerce platform. The conceptual model does not worry about how null values are handled, how indexes will be structured, or how the database will perform under load. The physical design, on the other hand, must address these practical constraints directly. For example, it defines that CustomerID cannot be null in the Order table (NOT NULL constraint), and that a ReorderLevel is set in the Inventory table to trigger restocking alerts. The physical model must also consider performance optimization through indexing (e.g., indexing ProductID in the Order table for faster lookups) and data integrity through referential integrity constraints. Thus, while the conceptual model answers the question of what data the business needs to represent, the physical model answers how that data will be safely and efficiently stored and accessed.

References

Groves, M. (2022, October 7). What is data modeling? Conceptual, physical, logical. Couchbase. https://www.couchbase.com/blog/conceptual-physical-logical-data-models/

Nalimov, C. (2021, September 22). What is a conceptual data model? With examples! Gleek.io. https://www.gleek.io/blog/conceptual-data-model.html

Vidhya, V., Jeyaram, G., & Ishwarya, K. (2016). Database management systems. Alpha Science International.

Ellie. (2022, July 6). Conceptual vs logical data models - What are the key differences? [Video]. YouTube. https://www.youtube.com/watch?v=conceptual-logical